OCP GLOBAL SUMMIT 2026 • TIME APPLIANCES PROJECT

AI 集群的时钟对齐遥测

Lukas Landgraf · Ahmad Byagowi，turbalance

时长 15 分 34 秒

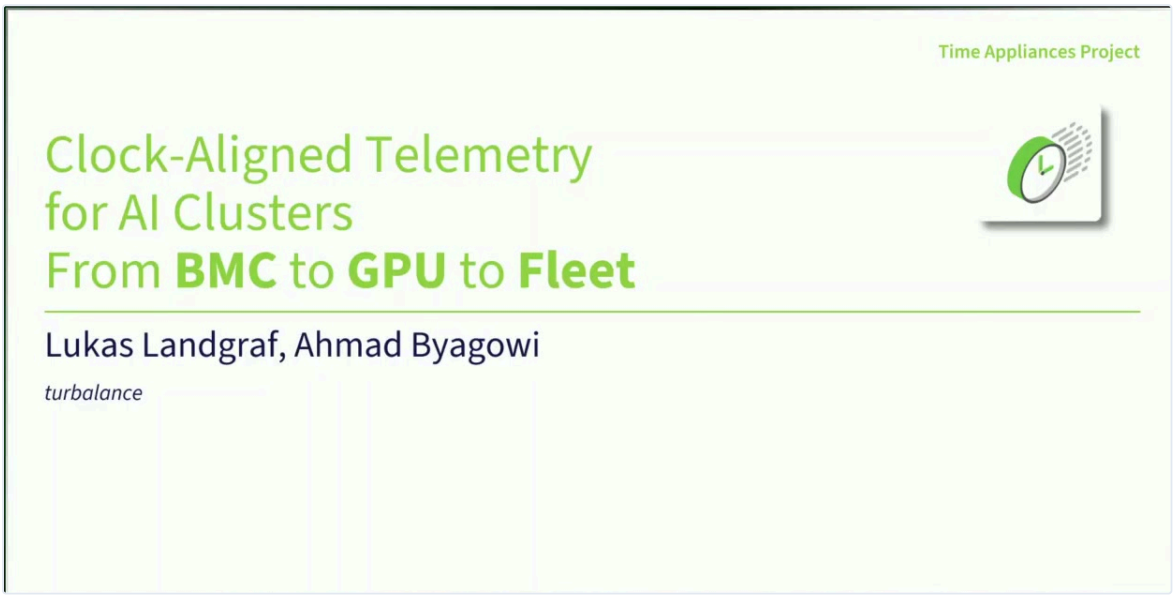
来源 youtube.com/watch?v=N_4_1orETrc

页数 16 页幻灯片

AI 集群的时钟对齐遥测：从 BMC 到 GPU 到集群

> OCP Global Summit 2026 · Time Appliances Project (TAP) 分会场　讲者：Lukas Landgraf、Ahmad Byagowi（turbalance）　全长 15 分 34 秒　|　共 16 页幻灯片

01　开场：从 BMC 到 GPU 到集群



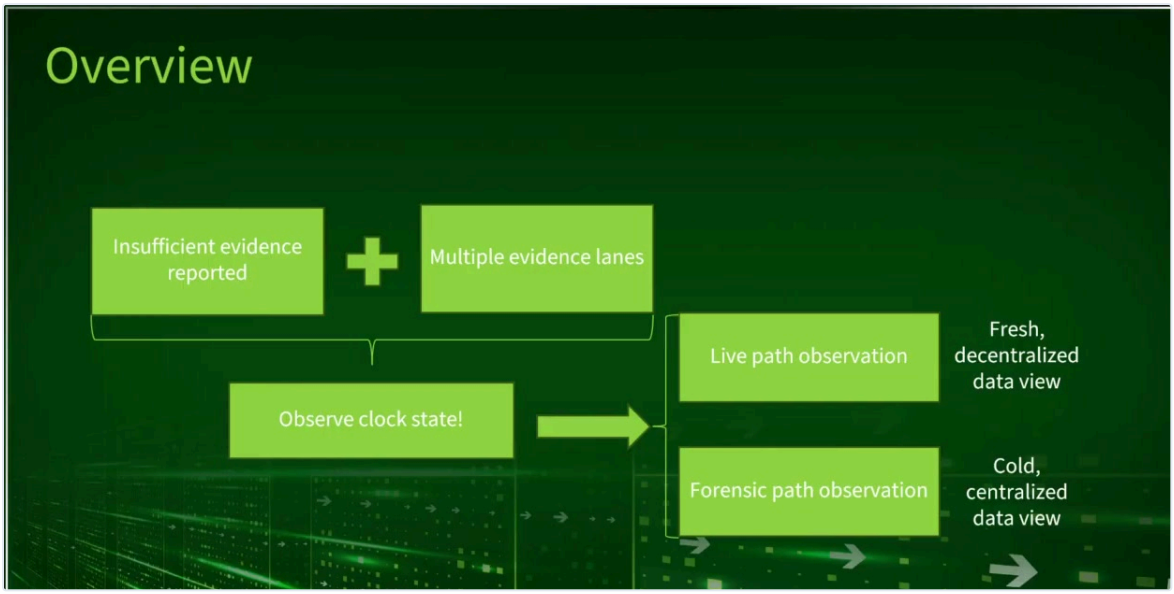
开场标题页：Clock-Aligned Telemetry for AI Clusters — From BMC to GPU to Fleet，讲者为 Lukas Landgraf 与 Ahmad Byagowi（turbalance）

好，这个——是的，好，酷。好，是的，正如主持人介绍的、也正如标题页所说，今天的演讲是关于 AI 集群的时钟对齐遥测，从 BMC 到 GPU 到集群。这到底是什么意思呢？呃，我们想知道在我们整个 AI 集群里发生的某些事件……

> "All right, this is yeah, okay, cool. Okay, yeah, as as announced, or as the as the title slide says, today's talk is about like clock aligned telemetry from AI for AI clusters from BMC to GPU to fleet. And what this exactly means is that um we want to know about like certain incidents in in in our entire AI"

> **小结**　议题是"遥测的时间维度"：不是再做一个监控面板，而是让来自 BMC、主机、GPU、网络、调度器的证据在同一个时间基准上可比。标题里的三个层级（BMC→GPU→Fleet）正是后续"六条证据通道"的跨度。

02　概览：证据不足 × 多条证据通道



Overview：证据不足（Insufficient evidence reported）加上多条证据通道（Multiple evidence lanes），指向"观测时钟状态"（Observe clock state!），再分成实时路径观测（新鲜、去中心化的数据视图）与取证路径观测（冷数据、中心化的数据视图）

呃，我们整个 AI 集群里。比如说你有一块 GPU 出了问题，你想知道问题出在哪、之前发生过哪些事件。你想收集一些证据。但很难找出哪些事件可能与你观察到的事件相关。所以我们想做的，首先是采集一些新鲜数据，作为分析的基础；第二，为了弄清到底是什么在相关，你需要去探究手上的数据。这就把我们带到了问题。

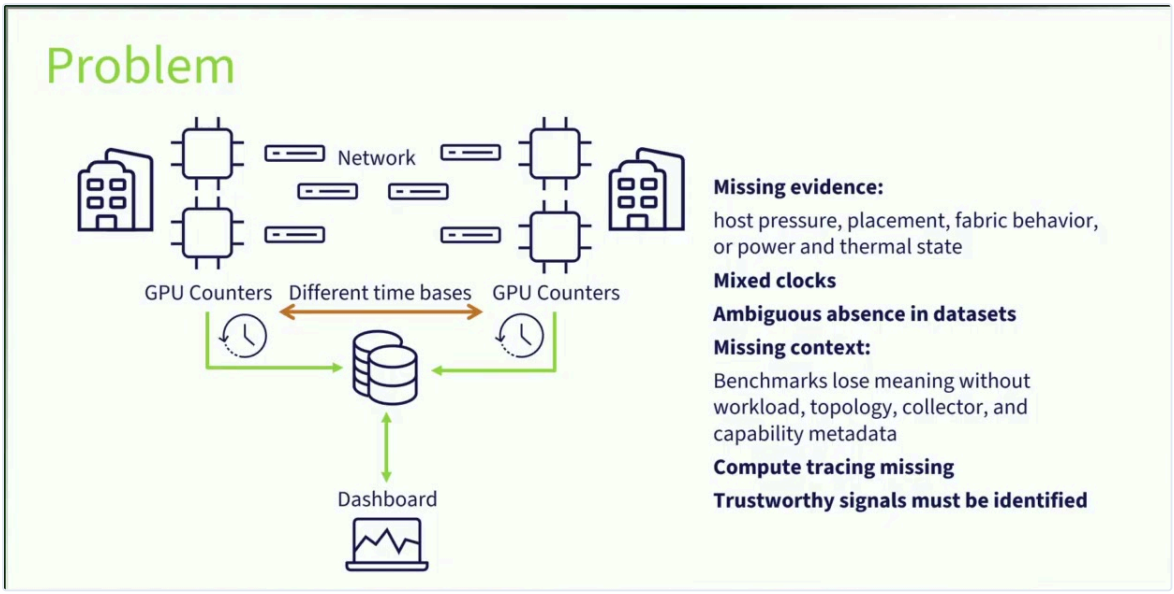
所以，呃，如果你不看时间，你就可能缺少证据。首先，你可能缺主机压力……

> "cluster. So, say that you you have a GPU and something's going wrong and you want to know what's what's the what's what's the issue for that, like what what events have been proceeding here. And you want to collect some evidence here. However, it's kind of hard to find out what what might be some events that might be correlating with the events that you have been observed there. And so, what we want to go for is that first of all, we want to collect some fresh data that uh that can first of all serve as a basis for analysis here. And the second thing is that um in order to find out what exactly might be correlating, you need to sort of investigate the data that you that you have. And so, this already brings us to the to the problem. So, um you will have some So, if you don't uh So, if you don't look at time, you might have some missing evidence. So, first of all, you you might be missing a host pressure"

> **小结**　论证起点：单一指标（如 GPU 计数器）无法解释事件，必须"多通道 + 时钟状态"两条腿走路。概览页把输出分成两条互补路径——实时（新鲜、去中心化）与取证（冷、中心化），二者共用同一份数据契约。

---

03 问题：缺少证据、混合时钟、语境丢失



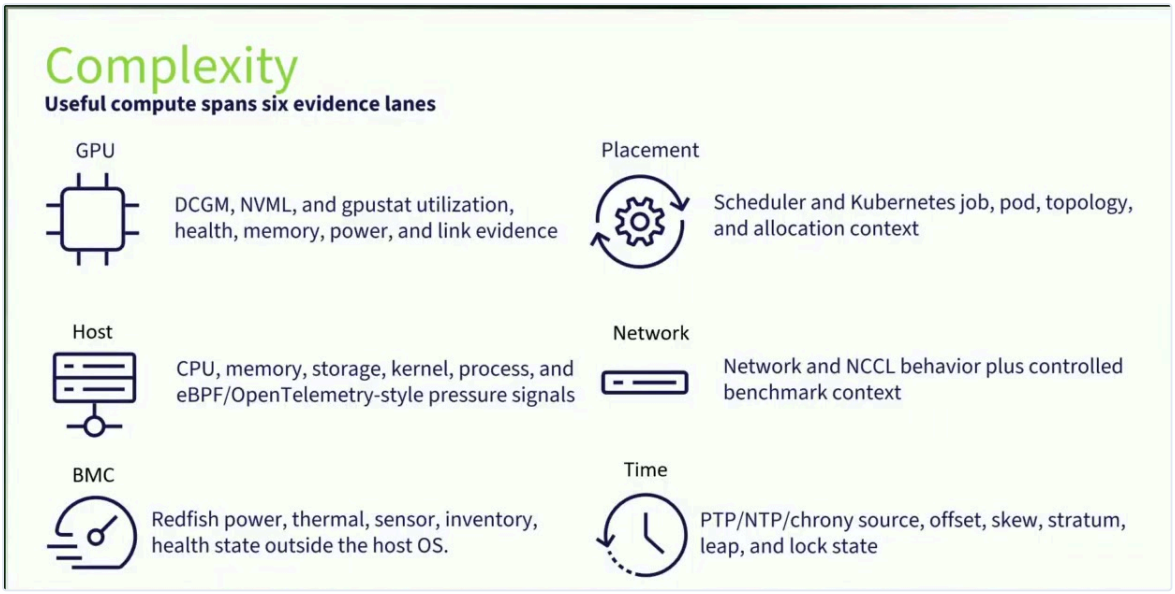
Problem：GPU 计数器分处不同时间基（Different time bases），网络与节点间缺乏对齐；右侧列出缺失证据（主机压力、放置、网络行为、功耗与热状态）、混合时钟、数据集中的模糊缺失、缺失语境（没有工作负载/拓扑/采集器/能力元数据则基准失去意义）、缺少计算追踪，以及必须识别可信信号

……放置、网络行为、功耗与热状态等等。如果你只看 GPU 计数器，你可能会漏掉某些时序行为。你可能会在数据集里遇到模糊的缺失，也可能丢失一些语境。所以，如果你不看工作负载，你手上的大量数据本质上就失去了意义。而且你还需要看网络拓扑、线缆元数据等等。另外，关于计算发生在哪里的追踪可能根本没有被采集。还有，你可能需要判断这些信号是否可信，才能看清数据中心里在发生什么。呃，这又被进一步复杂化，因为我们有……

> "placement, fabric behavior, power thermal state, and whatnot. If you if you if you um only look on at GPU counters, um you might be um you might be missing certain timing behavior. Um you might encounter some ambiguous absence in data sets, and you might also miss some context. So, if you don't look at the workload, a lot of the data that you have essentially becomes meaningless. And also, you need to look at the topology of a network, um cable metadata and whatnot. And also some tracing of like where you what compute has been happening might not be might not be collected. And also you might need to identify the signals if they are trustworthy or not in order to see what's happening in your data center. So, um this is further made complex by"

小结　五类具体缺陷：缺证据（主机/网络/功耗热）、混合时钟（不同时间基无法对齐）、数据集里的模糊缺失（无法区分"没数据"与"值为零"）、缺语境（工作负载/拓扑/采集器元数据）、缺计算追踪。共同后果是——数据看似齐全，却无法支撑因果判断。

04 复杂度：有用算力横跨六条证据通道



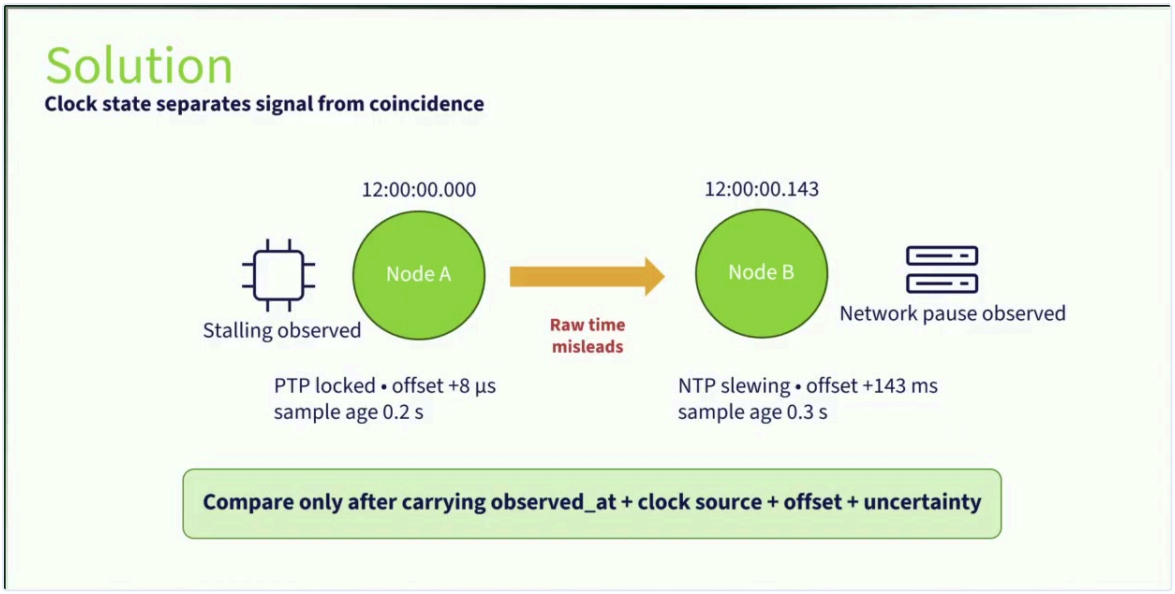
Complexity — Useful compute spans six evidence lanes：GPU（DCGM、NVML、gpustat 利用率/健康/内存/功耗/链路）、Host（CPU、内存、存储、内核、进程与 eBPF/OpenTelemetry 压力信号）、BMC（Redfish 功耗/热/传感器/清单/OS 之外的健康态）、Placement（调度器与 Kubernetes 作业/pod/拓扑/分配语境）、Network（网络与 NCCL 行为及受控基准语境）、Time（PTP/NTP/chrony 源、偏移、偏斜、层级、闰秒与锁定状态）

……我们拥有的六条证据通道。首先在 GPU 上，你可能想看 DCGM、NVML、gpustat 的利用率等等。你可能想从主机采集 CPU、内存、存储等方面的证据。从 BMC，你可以看 Redfish 的功耗、热统计、传感器清单等等。你要看你的工作负载放置——Kubernetes 作业以及东西是怎么摆的。然后网络与 NCCL 行为加上受控的基准语境，也是你可能想按时间关联起来的东西。最后，你可以把"时间"本身作为一个证据来源来采集。呃，那么，真正能拿到证据的解决方案是……

> "six evidence lanes that we have. So, first of all, on the GPU, you you might want to look at like DCGM NVML GPU stat utilization and whatnot. You might want to to collect evidence from from the host about like CPU, memory, storage, and so on and so forth. From a BMC, you can look at ref fish power, thermal stats, sensor inventory, and whatnot. You want to look at the placement of your workloads for your Kubernetes jobs and how stuff has been placed. And then the network and nickel behavior plus controlled benchmark context is something that's also something that that you might want to correlate with stuff that's happening on a timely basis. And finally, you can involve time as a as a source for certain evidence that you might want to collect. And um So, this the solution for actually"

小结 六条通道：GPU、Host、BMC、Placement、Network，加上 Time 本身。关键洞见是把"时间"当作第六条证据通道而非背景参数——只有时间自身的状态（源、偏移、偏斜、层级、锁定）被采集，前五条才可能被正确对齐。

05 方案：时钟状态把"信号"与"巧合"分开



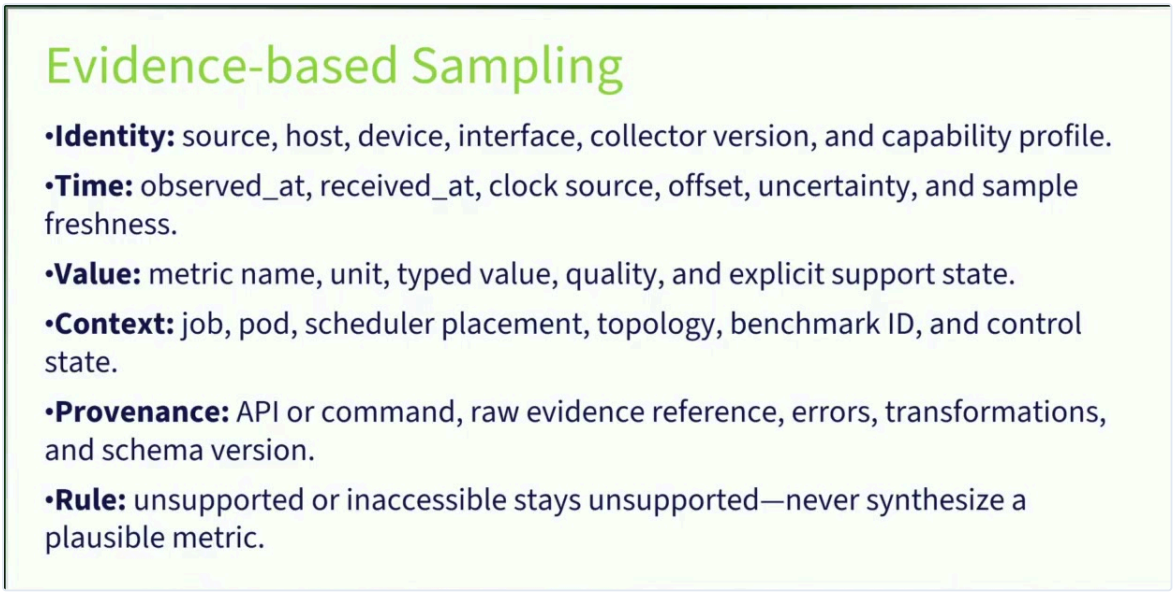
Solution — Clock state separates signal from coincidence：Node A 在 12:00:00.000 观测到停滞，PTP 锁定、偏移 +8µs、样本年龄 0.2s；Node B 在 12:00:00.143 观测到网络停顿，NTP slewing、偏移 +143ms、样本年龄 0.3s；结论：只有带上 observed_at + 时钟源 + 偏移 + 不确定度之后才能比较

……获取数据中心里发生之事的证据，其实就是采集更多数据——关于你拥有的事件、以及你时钟状态的数据。在这个例子里，你观察到某块 GPU 出现停滞；然后在另一个节点上观察到某处网络脉冲；而你的计时器可能有某个偏移，如果你不把时间因素算进去，就可能得出错误结论。所以，这就把我们带到"基于证据的采样"。呃，你之前看到的东西——你需要采取的措施——就是采集更多数据，所以……

> "getting some evidence about like the stuff that's that's been happening in your data center is actually to collect more more data about like the the events that you have and like the the state of your clocks. So, in this particular example, you have some stalling that you observe for for a GPU. And then you observe a network pulse somewhere on another node and you might have some certain offset for your timers and um your you might get some wrong conclusions if you if you don't account for the timing so and um so this brings us to evidence-based sampling so um you the the stuff that you saw saw before so the measures that you need to take um is to to gather more data so um"

小结 核心示例极具说服力：A 节点 12:00:00.000 的停滞与 B 节点 12:00:00.143 的网络停顿，看似相差 143ms；但 B 节点是 NTP slewing、偏移 +143ms，真实时刻几乎重合。规则由此确立：比较前必须携带 observed_at + 时钟源 + 偏移 + 不确定度。

06　基于证据的采样：六个字段



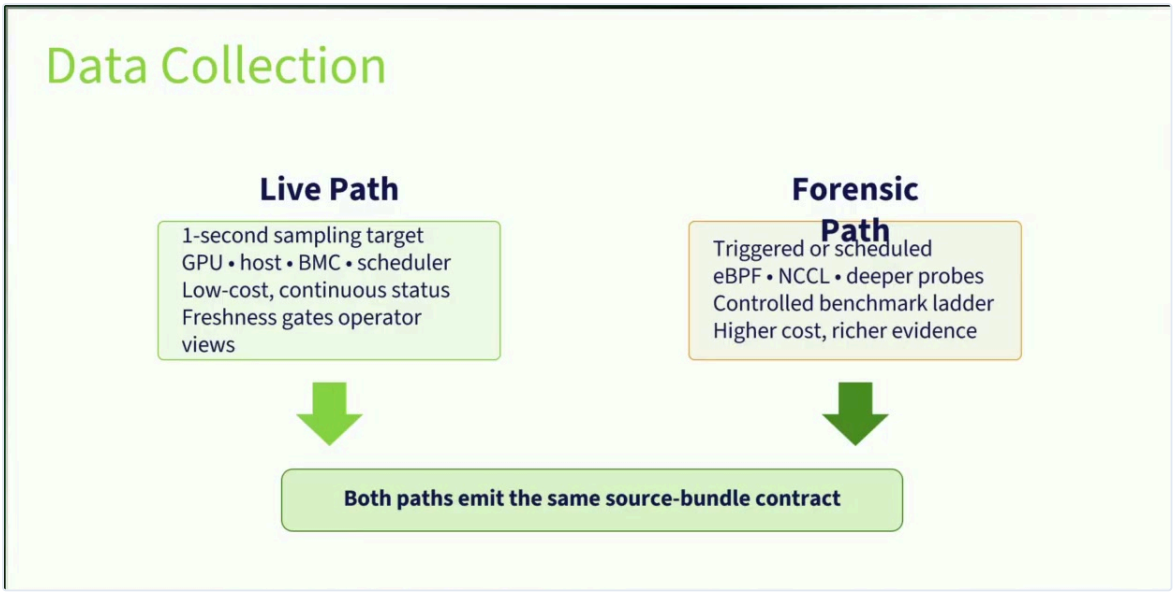
Evidence-based Sampling：Identity（源、主机、设备、接口、采集器版本、能力 profile）、Time（observed_at、received_at、时钟源、偏移、不确定度、样本新鲜度）、Value（指标名、单位、带类型的值、质量、显式支持状态）、Context（作业、pod、调度器放置、拓扑、基准 ID、控制状态）、Provenance（API 或命令、原始证据引用、错误、变换、schema 版本）、Rule（不支持或不可访问就保持"不支持"——绝不合成一个看起来合理的指标）

……你需要把来源考虑进来——主机、设备接口、采集器版本等等。关于时间，你需要看事件被观测到的时刻、被接收的时刻、时钟源、偏移、样本自身的不确定度与新鲜度；然后是值本身。呃，关于语境，你同样需要看真正在跑你 GPU 集群的那些东西；你还需要把它的来源（provenance）算进去——比如工作负载从哪来、这条命令是来自 API 还是别处。呃，所以如果你采集这类数据，你实际上就能拿到关于发生过之事的证据。为此你会走两条路径，所以……

> "you need to account for for the source the the host the device interface collector version and so on and so forth and regarding timing you need to look at the the time sense of when an event has been observed when has been received the the source clock the offset certain uncertainty in the sample freshness itself and then the values themselves as well and um so regarding the context again you need to look at the at the at the things that actually running your GPU cluster and you also need to account for the provenance of it so um like where where did workload come from was the command was like from an API or whatnot and um so if you collect this sort of data you can actually get some evidence about the the things that that have been happening and for that you will have two paths so"

小结　六字段构成一个"可审计样本"：Identity（谁报的）、Time（何时、以什么时钟、误差多少）、Value（值与其质量）、Context（在什么工作负载/拓扑下）、Provenance（经何路径、做过何变换）、Rule（不支持就写不支持，绝不编造）。其中 Rule 一条最具工程价值——它把"静默合成"这一常见反模式明确禁止。

07 数据采集：实时路径与取证路径



Data Collection：Live Path——1 秒采样目标、GPU/主机/BMC/调度器、低成本连续状态、新鲜度决定运维视图是否可用；Forensic Path——触发式或定时、eBPF/NCCL/更深探针、受控基准阶梯、成本更高但证据更丰富；两条路径输出同一份 source-bundle 契约

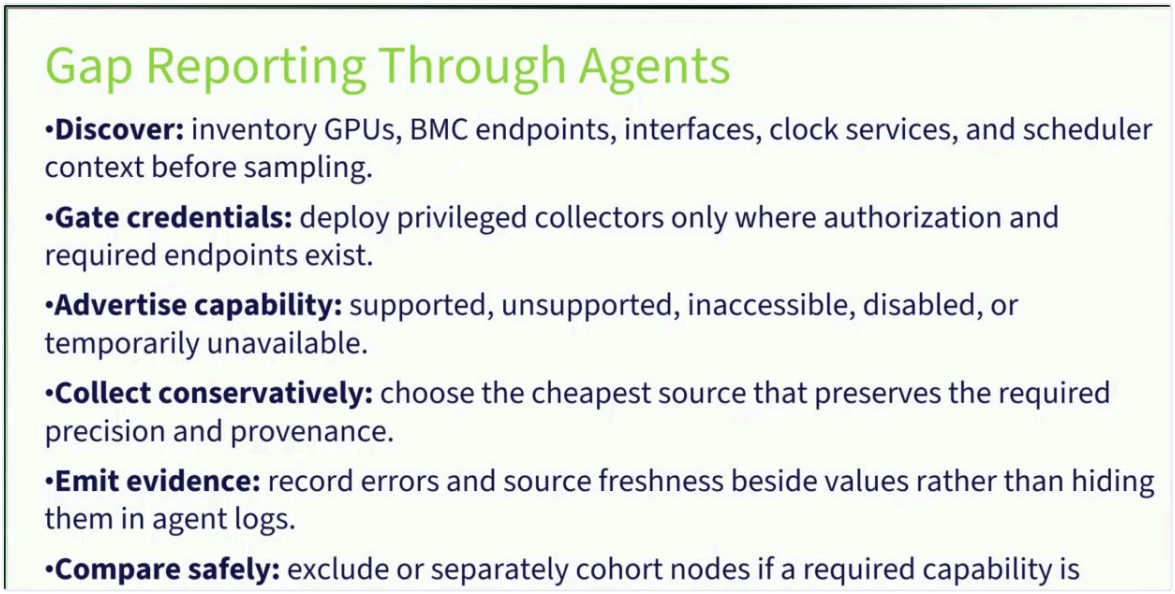
……实时路径就是纯粹在你的设备上、以及网络状态上做数据采样；第二条路径是取证路径，这条路径更贵一些，它需要被触发，你需要去翻查数据、看发生过什么，并采集某些证据。而这两条路径（输出）遵循同一份 source-bundle 契约。

> "the live path is just like pure pure data sampling on on your on your devices and the state of the network and then the second path is the forensic path and this path is a bit more uh is a bit more expensive so it needs to be triggered you need to you need to gather dig through the data what has been happening and there's some evidence that needs to be collected and so both of these paths with the same source bundle contract"

小结 两条路径的成本结构截然不同：实时路径以 1 秒为目标、低成本、连续，靠"新鲜度"决定运维视图是否可信；取证路径按需触发、用 eBPF/NCCL/更深探针，成本高但证据丰富。二者共用 source-bundle 契约，因此可以在同一分析面上拼接——这是"实时看趋势、取证查根因"能成立的前提。

---

08 通过代理上报缺口



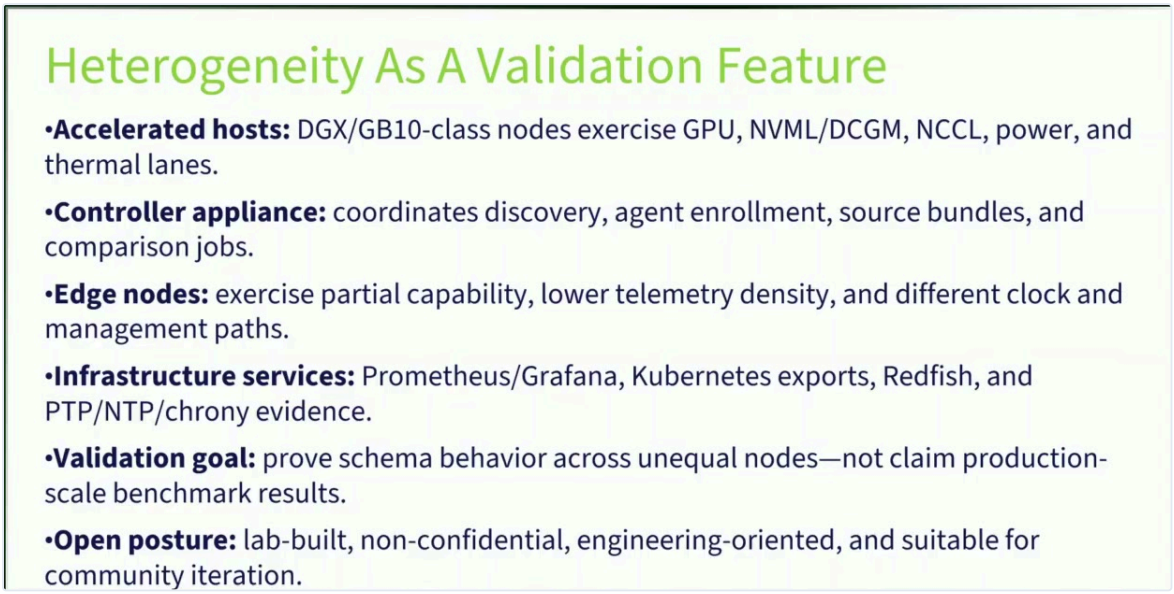
Gap Reporting Through Agents：Discover（采样前先盘点 GPU、BMC 端点、接口、时钟服务与调度器语境）、Gate credentials（只在有授权与端点处部署特权采集器）、Advertise capability（声明支持/不支持/不可访问/已禁用/暂时不可用）、Collect conservatively（选保留所需精度与来源的最便宜源）、Emit evidence（把错误与新鲜度和值一起记录，而不是藏在代理日志里）、Compare safely（缺少必需能力的节点应排除或单独分组）

……于是你基本上可以尝试通过代理把数据里的缺口上报出来，那需要被触发。呃，你可以先发现（discover）GPU、BMC 端点等等的清单；你可以拿到受门控的凭据（gate credentials）；你可以在设备上声明你所具备的某些能力。呃，你也可以保守地采集某些数据。你可以尝试通过那些代理发出证据，并安全地比较这些数据。是的。此外，我们在某些 GPU 集群里遇到的一个问题是，某些 GPU……

> "and so you can essentially try to report gas that you have in your data through through agents so that that needs to be triggered. Um so you can discover inventory GPUs with BMC endpoints and so on and so forth. You can gather the the gate credentials and you can advertise certain capabilities that you that you have on the the devices. And um also you can collect some certain data conservatively. Um you can try to emit the evidence that you through those agents and you can try to compare um those this data safely. And yeah. Furthermore, one problem that we have within within certain GPU fleets is that certain GPUs"

小结 代理的六条行为准则把"能力声明"提到一等公民：先发现、再门控凭据、然后显式声明支持状态（支持/不支持/不可访问/已禁用/暂不可用）。"保守采集"与"安全比较"两条尤其关键——缺能力的节点必须排除或单独分组，否则比较结果会被系统性污染。

09 异构性作为验证特性



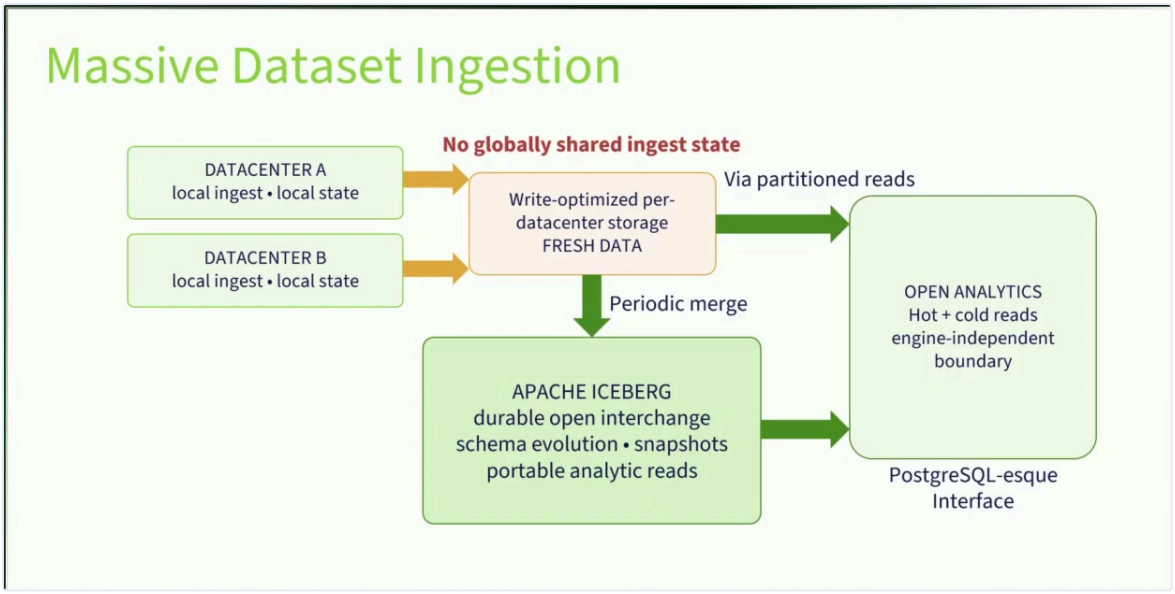
Heterogeneity As A Validation Feature：加速主机（DGX/GB10 级节点，覆盖 GPU、NVML/DCGM、NCCL、功耗与热通道）、控制器设备（协调发现、代理注册、source bundle 与比较作业）、边缘节点（部分能力、较低遥测密度、不同时钟与管理路径）、基础设施服务（Prometheus/Grafana、Kubernetes 导出、Redfish、PTP/NTP/chrony 证据）；验证目标是在不等的节点间证明 schema 行为，而非宣称生产级基准；姿态为实验室自建、非机密、面向工程与社区迭代

……可能与所宣称的节点"类别"并不相同。最终这些节点在质量上可能存在某些差异，而你可能想把这些识别出来。通过采集更多数据，你其实能查出这些 GPU 节点属于哪一类。呃，比如是不是来自不同的产线，而那些差异可能会、也可能不会表现为不同的故障行为。呃，是的。其他还包括控制器设备、作为节点的设备，以及基础设施服务；你也可以设定一些验证目标。呃，当真正开始采集数据时——我是数据库出身的……

> "might or might not be the same advertised well say class class of a node. And eventually there there might be certain differences in like the qualities of those those nodes and you might want to identify that. And by gathering more data you can actually find out what what kind of class those those GPU nodes were belonging to. Um so if it's like a different production line what not and those can or cannot might might or might not expose different failure behavior. Um yeah and other things include stuff like controller appliance um as nodes and infrastructure services and you can you can also go for some validation goals. Um when it comes down to actually gathering the data and so so I'm a database major"

小结 把异构性从"麻烦"翻转为"验证手段"：加速主机、控制器、边缘节点、基础设施服务四类不等的节点，恰好可以检验 schema 在不同能力密度与不同时钟路径下是否仍然成立。作者明确限定这是实验室自建、面向工程迭代的验证，而非生产级基准声明。

10　海量数据摄取：Iceberg 作为开放边界



Massive Dataset Ingestion：数据中心 A/B 各自本地摄取、本地状态（没有全局共享的摄取状态），写入优化的每数据中心存储承载"新鲜数据"，经分区读取提供给开放分析（热/冷读、引擎无关边界、类 PostgreSQL 接口），并周期性合并进 Apache Iceberg（持久、开放交换、schema 演进、快照、可移植的分析读）

……所以这是整个项目里我觉得最有意思的部分：你需要采集的数据量是巨大的，对吧？呃，而数据处理领域、或者说数据系统领域总体上，已经有一些创新。呃，所以当你有一个数据中心、想采集时间序列数据时，你最终可能就去用某个 Prometheus 数据库。呃，在我看来，那样是可以的。呃，但它有一些不便之处，比如它不支持 PostgreSQL 方言。呃，最重要的是，当你真正采集数据时，它是按 Prometheus 的方式存的，而不是以开放表格式（open table format）存的。

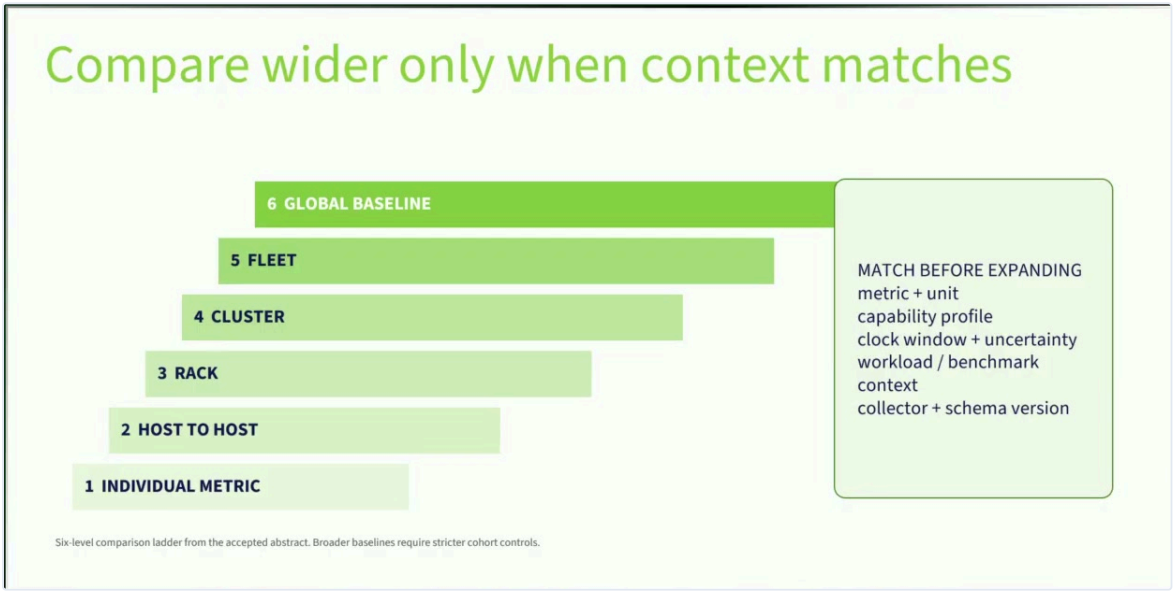
呃，而当你真正去采集数据、并想把它合并成某种大家都能理解、对数据结构有共识的东西时，你其实可能想用 Apache Iceberg 这样的开放表格式。呃，它可以为更广泛的社区充当一个数据存储，每个人都能用，可以加载进他们的 Snowflake 或 Databricks 流水线。呃，所以这里的整体提议是：对社区，我们提供某种写入优化的、按数据中心划分的存储，你可以装在自己的数据中心里，然后周期性地把这些采集到的数据合并进像 Apache Iceberg 这样的中央数据存储。呃，它可以作为开放分析的基础，支持热数据和冷数据读取。呃，你可以做到引擎无关，而你的整个边界就是这个 Apache Iceberg 数据存储。而且这在 schema 演进等方面也给了你一些灵活性。呃，不过这里最大的改进是：我们能拥有一个更大的数据集合，在其上操作、并识别出某些行为模式。

> "so this was the most interesting part of about this entire project for me um is that um, the data that you need to collect is kind of massive, right? And, um, there have been some, uh, innovations in the in the in the data processing space, or let's say like uh, data data systems uh, space in general. And, um, so when you have a data center and you want to collect a time series data, you might eventually just go for some Prometheus data database. And, um, in my opinion, that kind of works. Um, but, um, there are some certain inconveniences that you that you have for Prometheus or instance like it does not support, uh, the post uh, post SQL dialect. And, um, most of all, when you actually collect the data, it's like in the in the way that Prometheus collects it, but it's not like in an open table format. And, um, when you actually go for, uh, the collection of the data and you want to merge it down to, uh, to something that basically every everybody can understand where there's a common understanding of how the data has been structured, you might actually want to go for open table formats like, uh, Apache Iceberg. Um, that can serve for the broader part of the community actually as as some data store that, uh, that everybody can work with and that they can, um, say load to

> their, um, snow snow flake or data bricks pipeline. And, uh, so the whole proposal here is that, uh, for the community we would provide, um, um, some write optimized per data center storage that the that, uh, say you can you can install within your data center, and then you can periodically merge this data that you collected into, uh, a central data store like Apache Iceberg. Um, that can serve as like the basis for, uh, some open analytics for, uh, hot and cold data reads. Um, you can be engine independent, and, um, your your your whole boundary is just like this, uh, Apache Iceberg data store, um, that you have. And, this also gives you some some flexibility when it comes to like schema evolution and whatnot. Um, however, like the the biggest improvement that we would have here is to um have some uh larger collection of data where on which we could operate and where we could identify certain behavior."

> **小结**　存储架构的关键取舍：不设全局共享的摄取状态（避免跨站点写冲突），每个数据中心各自写优化本地库，再周期性合并到中央 Iceberg。Iceberg 承担"持久、开放交换、schema 演进、快照"的角色，热存则留给实现自选。收益不是更快的查询，而是"更大的可比数据集"——这才是能识别行为模式的前提。

11　仅在语境匹配时才扩大比较范围



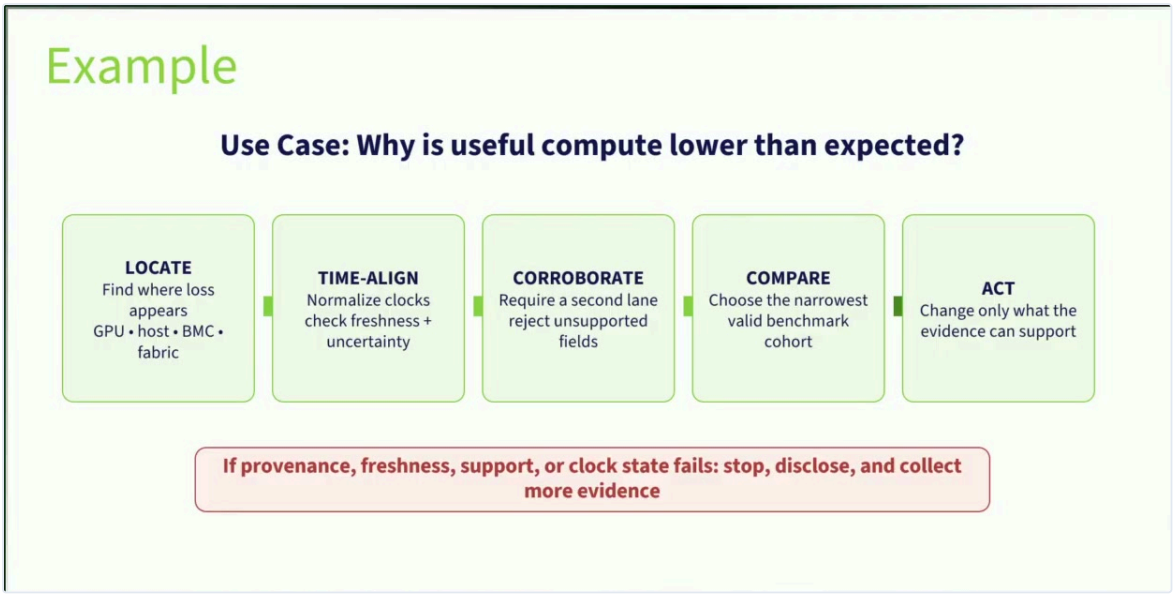
Compare wider only when context matches：六级比较阶梯——1 单指标 → 2 主机对主机 → 3 机架 → 4 集群 → 5 集群整体 → 6 全局基线；右侧列出扩大范围前必须匹配的项：指标与单位、能力 profile、时钟窗口与不确定度、工作负载/基准语境、采集器与 schema 版本

那么，呃，关于证据，你基本上只需看所采集数据的语境，并根据语境去把它关联到数据中心的某些部分。

> "So, um regarding the the evidence um you would you would essentially just um just look uh about the on the context of of of the of the data that you've been collecting. And you would essentially try to correlate that to to certain parts of your um of of your uh of your data center uh depending on the context. And so, um for instance, um you could try to look at uh the the case that uh maybe your compute is lower than uh than what you might have been expecting."

> **小结**　六级阶梯（单指标→主机对→机架→集群→集群整体→全局基线）规定的是"可比范围"，而不是"分析深度"：范围每扩大一级，就要求更强的一致性（指标与单位、能力 profile、时钟窗口与不确定度、工作负载语境、采集器与 schema 版本）。换句话说，比较范围的上限由语境匹配度决定。

12　示例：为什么有用算力低于预期



Example — Use Case: Why is useful compute lower than expected?：LOCATE（找出损失出现在哪：GPU/主机/BMC/网络）→ TIME-ALIGN（归一化时钟、检查新鲜度与不确定度）→ CORROBORATE（要求第二条通道、拒绝不支持的字段）→ COMPARE（选最窄的有效基准队列）→ ACT（只改证据支持的东西）；若来源、新鲜度、支持状态或时钟状态任一失败：停止、披露、并采集更多证据

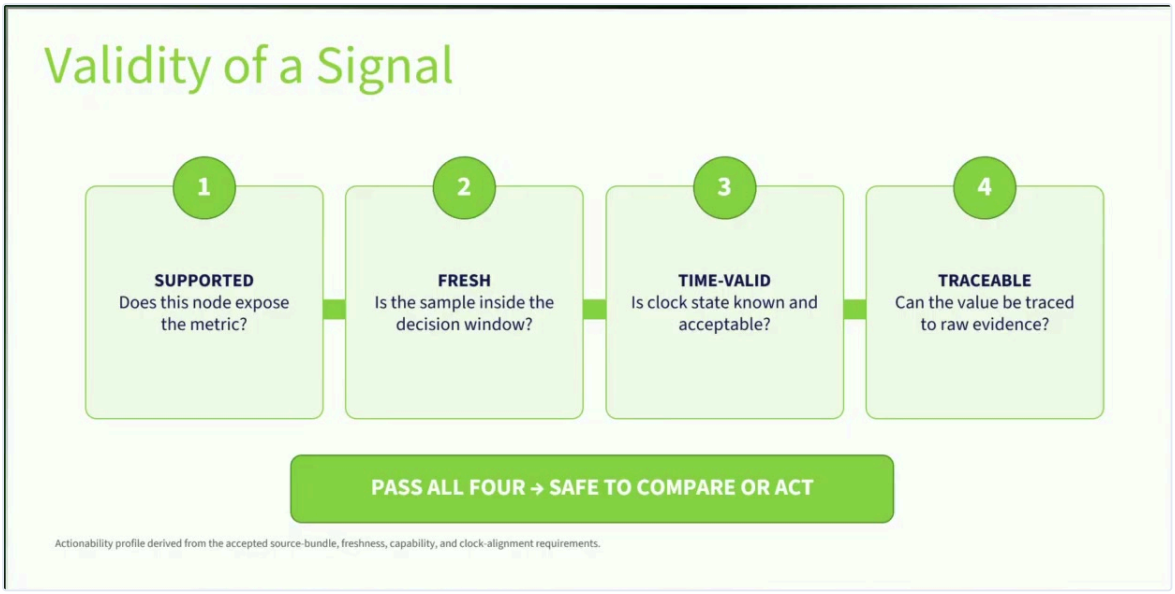
所以，呃，举例来说，你可以去看这样一种情况：你的算力可能低于你的预期。那么你会先尝试定位损失出现在哪里——可能在 GPU 上、主机上，你可以通过 BMC 去识别，或者它其实就在网络本身上。

呃，然后你会尝试对齐你的时钟，检查时钟的新鲜度和某些不确定度。接着你会去做佐证（corroborate），看看是否有某些字段并不被支持。然后你会尝试比较你的数据，去验证你认为可能发生的事件是否真的在发生，还是说它只是一个相关而非因果。最后，你可以针对你看到的事件采取行动——根据你实际采集到的证据，采取某些措施、改动参数，或在数据中心里做某些设置。

> "for instance, um you could try to look at uh the the case that uh maybe your compute is lower than uh than what you might have been expecting. So, you would first try to locate where uh where the loss appears. That might be um on the GPU, the host, um uh you could try to identify via the BMC or like um could be just located on the fabric itself. And um then you will try to align your clocks um and check about like the freshness and certain uncertainty um in in your clocks. And then you would go over to corroborate um where you would look if you if there are some certain fields that that aren't supported. And then and then you would try to compare uh your your data for the for the evidence about like what what the event that you might think that might have happened. um is actually happening or like if it's just something that's just like a correlation but not really a causation uh thing. And then at the end you can try to act upon the uh the event that you that you saw. And um you can try to take certain measures and change um uh parameters or set up something in your data center depending on the evidence that you actually gathered."

小结　五步工作流把"因果"拆成可执行动作：定位 → 对齐时钟 → 跨通道佐证 → 在可比范围内比较 → 只改证据支持的参数。其中"佐证"这一步专门用来排除假相关；底部的失败处理规则更关键——来源/新鲜度/支持状态/时钟状态任一不成立，就停止、披露、再采集，而不是继续推断。

13　信号的有效性：四道关



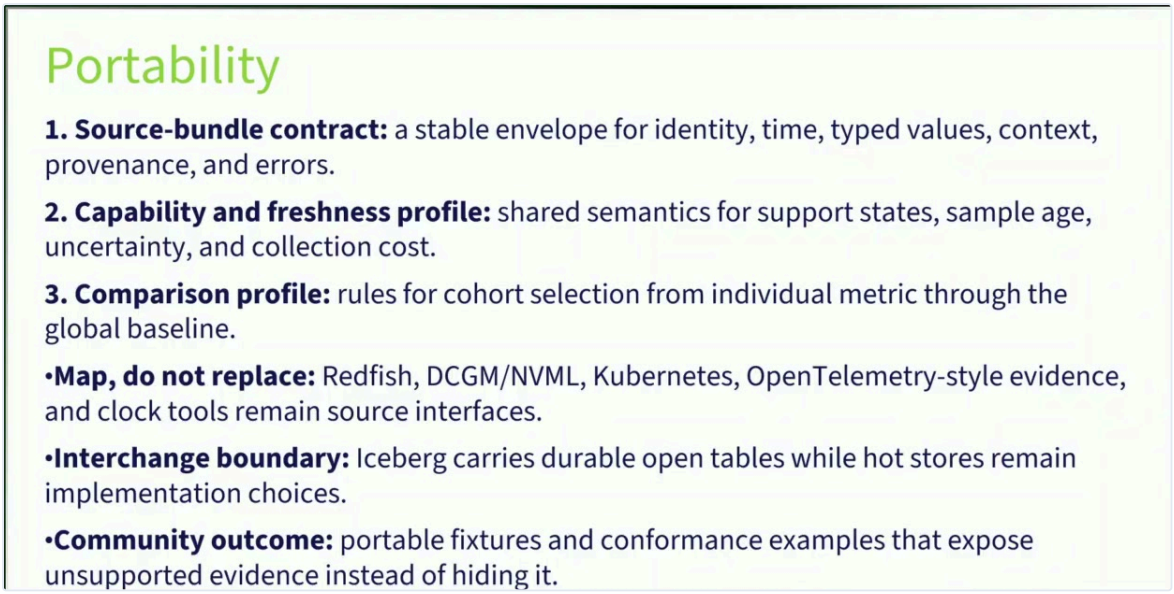
Validity of a Signal：①SUPPORTED（该节点是否暴露此指标）→ ②FRESH（样本是否落在决策窗口内）→ ③TIME-VALID（时钟状态是否已知且可接受）→ ④TRACEABLE（该值能否追溯到原始证据）；四关全过才"可以安全比较或行动"

……那么，关于信号的有效性，你也可以用分步的方式来处理。你先看你要看的那个节点是否真的暴露了该指标。呃，你可以看样本是否落在精度窗口内——也就是它是否满足你关心的时间上的新鲜度标准。以及时钟状态本身是否有效、可接受。最后你可以看这个值本身是否可追溯。如果你看到所有这些有效性标准都被满足，你其实就可以说：好，它是安全的，可以比较，也可以据此行动。

> "you could also handle this in a stepped fashion. So you you go over to look if if a node that you you are looking at actually exposes the metric. Um you could look if the sample inside is inside precision window so if it meets your freshness criteria regarding the the time that you look at. And if if the clock state itself is is valid and acceptable. And at the end you can look if the value itself is actually traceable or not. And if you if you see all of these validity criteria being met you can actually say that okay it's safe and compare to to compare or act upon it."

小结　四道关是有序的短路判断：支持 → 新鲜 → 时钟有效 → 可追溯，任一失败即不进入下一关。这一设计把"这个数能不能用"从经验判断变成可枚举的检查表，也让"不可用"成为一个明确的、可上报的状态，而不是静默的空值。

14　可移植性：三份契约



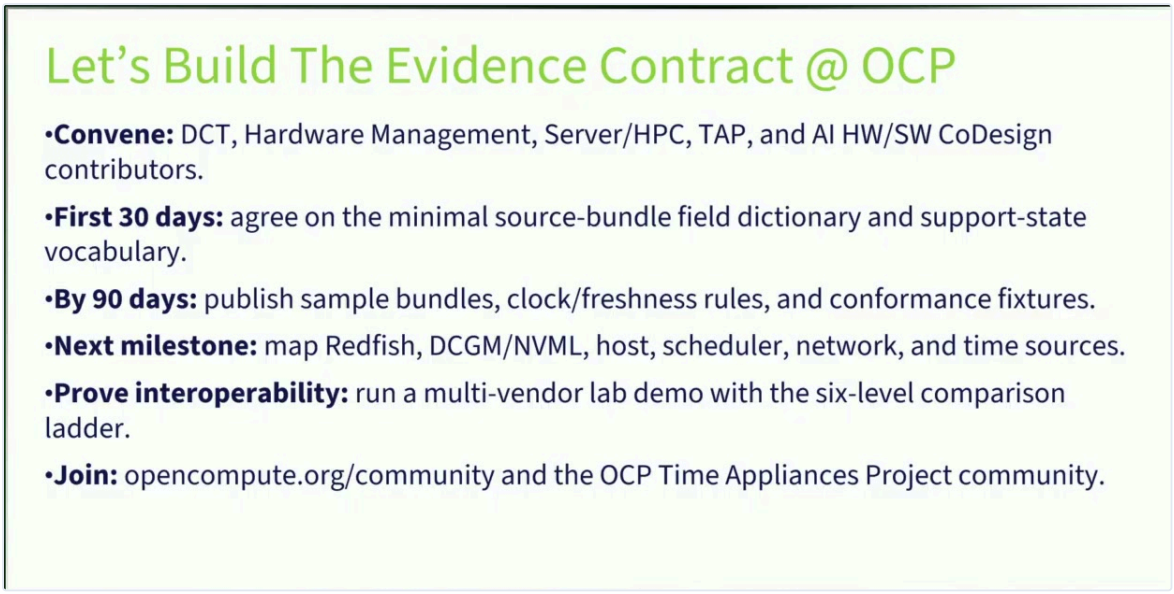
Portability：①Source-bundle 契约——身份、时间、带类型的值、语境、来源与错误的稳定信封；②能力与新鲜度 profile——支持状态、样本年龄、不确定度与采集成本的共享语义；③比较 profile——从单指标到全局基线的队列选择规则；以及三条姿态：映射而非替换（Redfish/DCGM/NVML/Kubernetes/OpenTelemetry 与时钟工具仍是源接口）、交换边界（Iceberg 承载持久开放表，热存留作实现选择）、社区产出（可移植夹具与一致性示例，暴露不支持而非隐藏）

那么，关于可移植性，我们有三份契约。首先是一份 source bundle 契约，它是身份、时间、带类型的值、语境、来源与错误的稳定信封。然后我们有能力与新鲜度 profile。接下来我们还有一份比较 profile。这些契约帮助我们映射（map）而非替换 Redfish、DCGM、NVML、Kubernetes 以及 OpenTelemetry 风格的证据——也就是我们从两个源采集到的数据。它也设定了一个交换边界，使用我们自己的 schema、我们自己的数据源。呃，Iceberg 承载持久的开放表，而热态存储本身可以保持为实现选择，取决于你想要什么、以及你想多快写入持久数据。而对社区来说，这样一个数据存储的产出将是可移植的夹具（portable fixtures）和一些一致性示例——你知道，它们暴露某些不支持的证据，而不是把它藏起来。这样你就真的能从采集到的数据中推断出某些结论。是的，那么这里的行动号召是……

> "So regarding the portability we have three contracts. So first of all we have a source bundle contract which is a stable envelope for identity time and type values, context, provenance, and errors. And finally we have the capability and freshness profile. And next thing is also we we have a comparison profile. And so these these contracts help us to to map and not replace things like Redfish, DCGM, and NVML, Kubernetes, and OpenTelemetry style evidence of the data that we collected from both sources. And it also sets an interchange boundary using our our our scheme our our data source that we have. So um Iceberg carries a durable open tables while the hot state store itself can remain implementation choice or not depending on what you want for your data center and how how how fast you want to actually write your durable data. And the outcome for the community of such a data store would be portable fixture and some conformance example is you know, exposes some unsupported evidence instead of hiding it. So, you'll be able to actually infer some some certain findings from the data that you're collecting. Yeah, so the call to action here is to"

> **小结**　三份契约分工明确：source-bundle 定义"一条样本长什么样"，能力与新鲜度 profile 定义"什么算可用"，比较 profile 定义"谁能和谁比"。"映射而非替换"是这套设计的政治基础——Redfish/DCGM/NVML/K8s/OpenTelemetry 继续作为源接口存在，新契约只在其上叠加一层可互换的信封。

15　在 OCP 共建证据契约



Let's Build The Evidence Contract @ OCP：Convene（DCT、硬件管理、Server/HPC、TAP、AI 软硬件协同设计的贡献者）；First 30 days（就最小 source-bundle 字段字典与支持状态词汇达成一致）；By 90 days（发布样例 bundle、时钟/新鲜度规则与一致性夹具）；Next milestone（映射 Redfish、DCGM/NVML、主机、调度器、网络与时间源）；Prove interoperability（多厂商实验室演示六级比较阶梯）；Join（opencumpute.org/community 与 OCP TAP 社区）

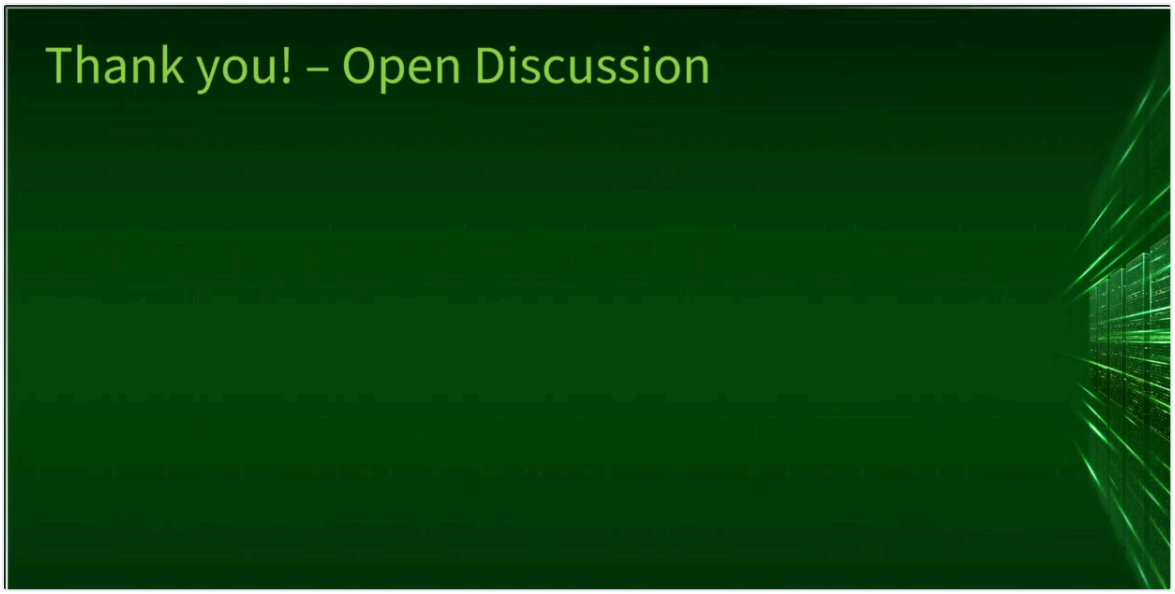
……真正走到一起，把这份设计定稿。也就是在 DCT、硬件管理、服务器、TAP，以及 AI 硬件软件协同设计上召集各方。一个提议是先做出一个最小可用产品：source bundle 字段字典与支持状态词汇。在接下来的 90 天里，我们可以尝试就发布样例 bundle 以及一些时钟与新鲜度规则达成一致。是的，我们也可以进一步讨论里程碑；而最重要的大概是证明不同集群之间的互操作性。是的，所以这就是一个加入社区的号召，加入 OCP 时间设备项目委员会，这样这件事不会半途而废，我们才能真正推进这个项目。是的，那么，谢谢各位，我们可以进入讨论了。

（主持人） 太好了。谢谢 Lucas。观众有任何问题吗？

> "to actually come together and finalize this this design. So, to to convene on DCT the hardware management the servers TAP and the AI hardware software code design and uh one proposal would be to to gather a minimum minimal working product for for the source bundle field dictionary and support state vocabulary. And within the 90 the next 90 days we could try to agree to publish the sample bundles and some clock and freshness rules. And yeah, we can also discuss further milestones about this and the most important thing is probably to prove the interoperability between different fleets and yeah. So, this would be a call to to join the community. And the OCP time appliances project committee such that this won't just fizzle out and we can actually work on on this project. So, yeah, with that I would like to thank you and yeah, we could try to go over to the discussion. Awesome. Thanks, Lucas. Any questions from the audience?"

小结　路线图有明确的时间盒：30 天定字段字典与支持状态词汇，90 天发样例 bundle 与时钟/新鲜度规则，下一里程碑做源映射，最终以多厂商实验室演示六级阶梯来证明互操作。起点刻意压到"词汇表"这一层——先让大家对"什么叫支持/新鲜/可追溯"有共同语言，再谈实现。

16　致谢与开放讨论



结尾页：Thank you! – Open Discussion

好。呃，如果没有（问题），那就谢谢 Lucas。是的，进入下一个演讲。

> "Okay. Uh if not then thanks Lucas, I guess. Yeah, move to the next presentation."

> **小结**　本场无提问，直接进入下一场。整场的主张可以压缩为一句话：AI 集群的可观测性瓶颈不在采集量，而在"证据是否带时间语境、是否声明支持状态、是否可追溯"——这三件事需要一份跨厂商的证据契约来固化。

附录　时间轴

时间	内容
00:00	开场：从 BMC 到 GPU 到集群
00:24	概览：证据不足 + 多通道 → 观测时钟状态
01:18	问题：缺证据、混合时钟、缺语境
02:27	六条证据通道
03:27	方案：时钟状态区分信号与巧合（+143ms 示例）
04:06	基于证据的采样六字段
05:12	实时路径与取证路径
05:45	通过代理上报缺口

时间	内容
06:30	异构性作为验证特性
07:30	海量摄取：本地写优化 + Iceberg 中央合并
09:51	仅在语境匹配时扩大比较
10:21	示例：有用算力低于预期的五步排查
11:45	信号有效性四关
12:30	可移植性三契约
13:57	行动号召与路线图
15:06	结束（无提问）